REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Ricardo Ruz

9 de abril de 2026

21:25

1. Entorno experimental

En este experimento se utilizó un computador con las siguientes características: procesador Intel Core i5 1235U (10 núcleos, hasta 4.4 GHz), 12 GB de RAM DDR4 3200 MHz, Windows 11 con WSL2 (Ubuntu 24.04 LTS) y compilación con g++ 12.4.0.

Para evaluar el rendimiento, se trabajó con los datasets descritos en el enunciado y se generaron 3 instancias por combinación.

- **Arreglos unidimensionales:** tamaños $n \in \{10, 10^3, 10^5, 10^7\}$, con entradas ascendentes, descendentes y aleatorias, y valores en los dominios $D1$ (de 0 a 9) y $D7$ (de 0 a 10^7). Se comparó Quick Sort (pivote aleatorio y partición de Hoare), Merge Sort y std::sort.

- **Multiplicación de matrices:** pares de matrices cuadradas con tamaños $n \in \{2^4, \ldots, 2^{10}\}$, con estructuras densa, diagonal o dispersa, y dominios $D0$ y $D10$. Se compararon los algoritmos Naive y Strassen.

A excepción de std::Sort, todo algoritmo implementado fue obtenido en la página de Geeks For Geeks

2. Resultados y Análisis

2.1. Ordenamiento de arreglos

Al ordenar los arreglos unidimensionales, con los algoritmos Quick sort, Merge sort y la función de sort propia de C++, se obtuvieron los siguientes resultados en la muestra de 1000 elementos:

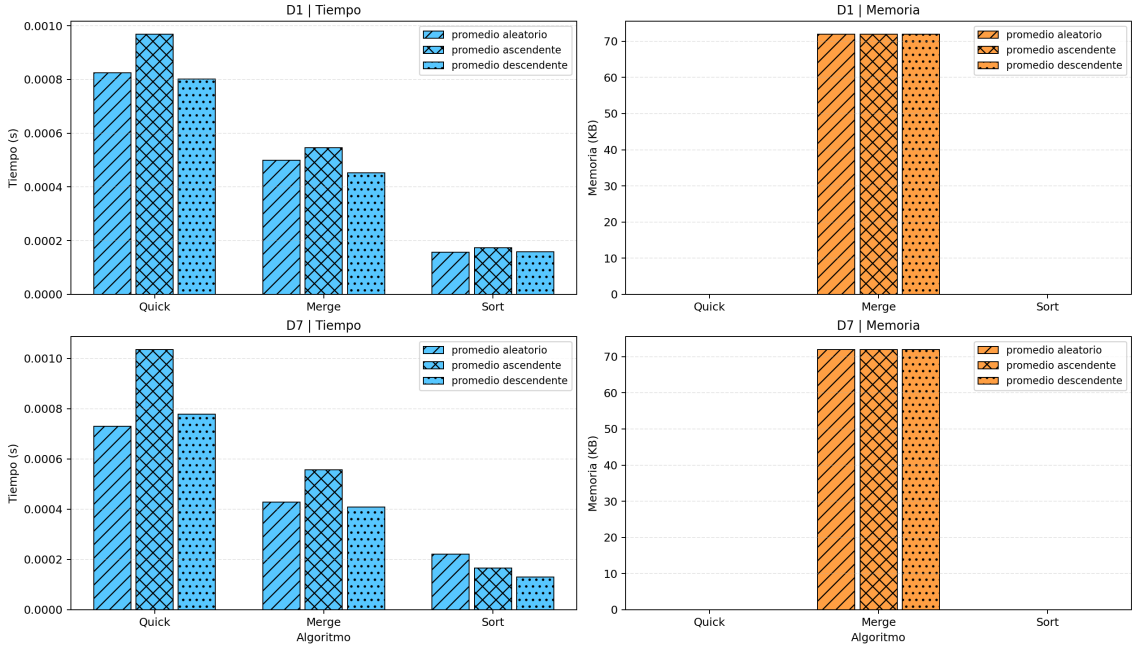

Figura 1: Algoritmos de ordenamiento lineal (promedio de los 3 casos) para tamaño n =1000

Cómo se puede ver, para un tamaño de muestra de 1000 elementos, el tiempo de resolución de los elementos es parecido para los casos ascendentes, aleatorios y descendentes, mientras que Quick se demoró considerablemente más que Merge y Sort. En cambio, el uso de memoria de Quick y Sort fueron prácticamente nulos, mientras que Merge usó en todo caso alredero de 70Kb de memoria.

Esto es acorde al tiempo y espacio aproximados de cada algoritmo, ya que los 3 algoritmos tienen un tiempo promedio $O(nlogn)$, pero el cambio radica en el peor tiempo de ejecución, ya que Quick es $O(n$š$)$, mientras que Merge y std::Sort son $O(nlogn)$

Para el tamaño, ya que std::Sort y Quick son algoritmos prácticamente in-place, se ve que la memoria usada es casi nula, mientras que Merge usa un espacio de $O(n)$

Debido a que para mayor cantidad de elementos, este comportamiento se mantiene, se omitirá habla de mayores tamaños.

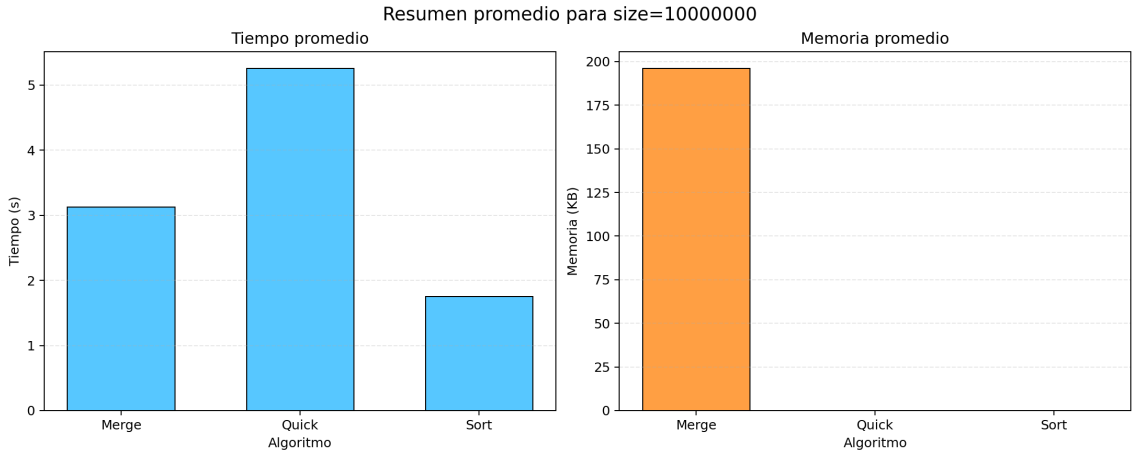

Figura 2: Algoritmos de ordenamiento lineal (promedio de los casos) para tamaño n =10000000

Para una muestra de 10000000 de elementos, se observa que el orden en tiempos de ejecución se sigue manteniendo, mientras que la memoria usada por Merge subió a los 250KB, en std::Sort aumentó de forma leve y que Quick sigue siendo in-place.

Esta es una fiel representación de la descripción de cada algoritmo, ya que Quick Sort demora más pero casi no usa memoria, Merge demora menos que Quick, a cambio de usar una memoria igual a la cantidad de elementos, y std::Sort está constantemente buscando cuál es el mejor subalgoritmo para seguir, por lo que siempre tendrá mejores resultados que los otros dos.

2.2. Multiplicación de matrices

Al multiplicar las matrices con los algoritmos Naive y Strassen, se obtuvieron los siguientes resultados para n = 256, osea, 2 matrices de 256 elementos cada una:

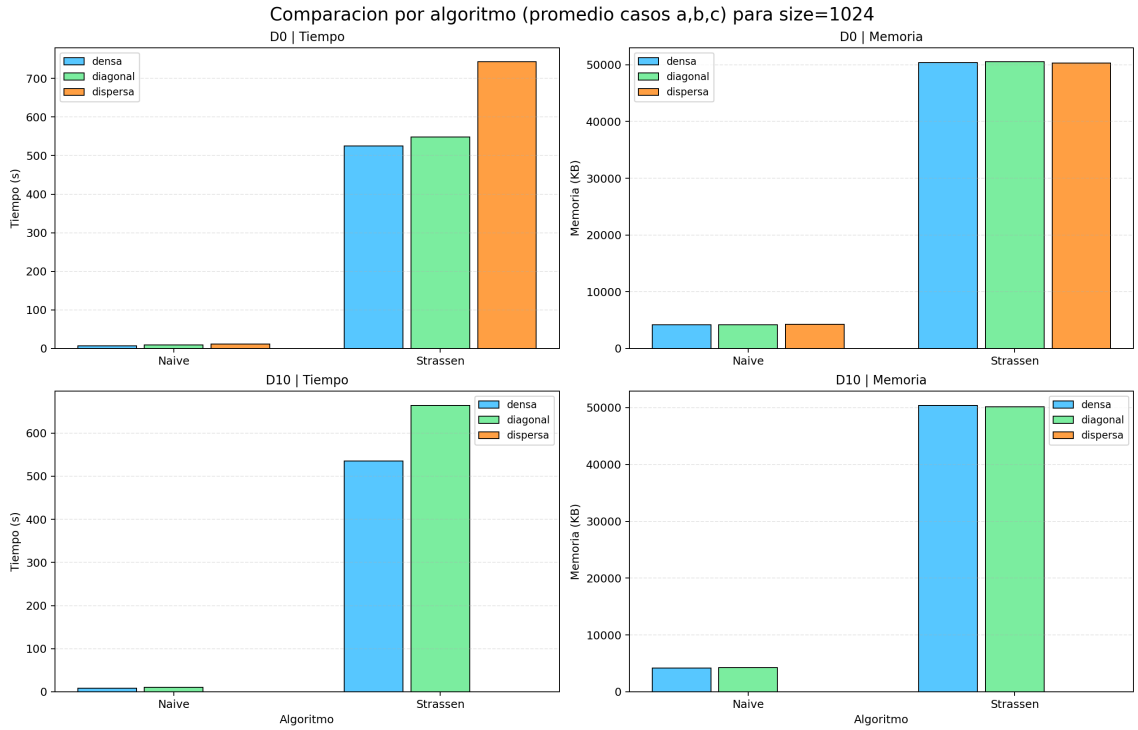

Figura 3: Algoritmos de multiplicación de matrices (promedio de los casos) para tamaño n =1024

En la tabla, se puede observar que Naive usa mucho menos tiempo y espacio que Strassen, aunque el primero sea $O(n$ş$)$ y el segundo sea $O(n^{2,87})$, lo que se contradice.

Los tiempos en Naive van en aumento lineal entre densa, diagonal y dispersa, lo que hace sentido debido a que cada vez sube más la cantidad de elementos a procesar, mientras que en Strassen hay un salto significativo en la diagonal y la dispersa, ya que aquí, la diferencia de elementos no nulos con Strassen hace que se realice mucho más esfuerzo.

En cuanto a espacio, se puede ver que tanto para D0 como D10, se obtiene un uso similar entre casos, ya que el algoritmo de Strassen realiza múltiples copias de las submatrices durante la recursión, lo que resulta en un consumo de memoria de $O(n$š$)$ independientemente de la densidad de la matriz. Por el contrario, Naive mantiene un uso de memoria prácticamente constante, ya que solo almacena la matriz resultado sin crear estructuras intermedias adicionales.

El mismo comportamiento se ve para matrices más pequeñas, por lo que se ignorarán. Además, debido al tiempo de ejecución, no se realizó el caso de prueba dispersa para D10, pero se asume que tendría un comportamiento parecido al de D0.

La contradicción de tiempo sólo se puede justificar con una mala implementación del algoritmo de Strassen, ya que por ejemplo, no recibe la matriz cómo referencia, si no que la recibe cómo parámetro, incrementando de forma exponencial el tiempo y la memoria utilizados.

3. Conclusiones

La extensión máxima para esta sección es un párrafo breve (máximo 1/4 de página).

En base a los resultados obtenidos, se observan las siguientes cosas:

- Para el ordenamiento de arreglos el comportamiento de los algoritmos se mantiene consistente para los distintos tipos de entrada: los tiempos de ejecución son similares entre los casos ascendentes, aleatorios y descendentes, pero Quick Sort se demora considerablemente más que Merge Sort y std::sort, mientras que en memoria Merge utiliza más espacio y los otros dos se mantienen básicamente in-place. Esto coincide con las cotas teóricas esperadas, donde el tiempo promedio es $O(n \log n)$, pero Quick puede caer a $O(n^2)$, y Merge requiere espacio adicional de $O(n)$.

- Para multiplicación de matrices, los gráficos muestran una contradicción con la teoría, ya que Naive termina usando menos tiempo y memoria que Strassen aun cuando sus órdenes son $O(n^3)$ y $O(n^{2,87})$ respectivamente; la explicación más realista es una implementación defectuosa de Strassen (por ejemplo, copias innecesarias y un manejo ineficiente de submatrices), por lo que debe ser revisada y validada con matrices mucho más grandes, donde el overhead deje de dominar y Strassen pueda opacar por completo a Naive.